

IBM App Connect

Advanced Integration Lab — Lab 2

Medical Claims & Patient Enrichment System

External SOAP/REST Calls · ITX Caching · Retry Logic · Message Expiry · DB2

6 Message Flows

External API/SOAP

ITX Caching

Retry + Expiry

Version 1.0 · April 2026

Pre-requisite: IBM App Connect Lab 1 (Patient Appointment System)

Estimated duration: 5 – 7 hours · 17 Parts · 6 Flows

Table of Contents

Part 1	Lab Overview & Use Case
Part 2	Architecture & Data Flow
Part 3	External Services — Mock Setup
Part 4	Database Schema
Part 5	IBM MQ Queue Design
Part 6	ITX Shared Cache Configuration
Part 7	WSDL — Insurance Validation Service
Part 8	OpenAPI — Patient Lookup Service
Part 9	Flow 1 — Appointment Intake Flow
Part 10	Flow 2 — Doctor Availability Flow (REST Lookup)
Part 11	Flow 3 — Patient Enrichment Orchestrator (Cache + REST + SOAP)
Part 12	Flow 4 — Insurance Validation Flow (External SOAP)
Part 13	Flow 5 — Retry & Message Expiry Flow
Part 14	Flow 6 — Claims Submission Flow (External SOAP)
Part 15	Cache Design & ESQL Patterns
Part 16	Retry & Expiry ESQL Patterns
Part 17	End-to-End Testing & Test Matrix

Part 1 — Lab Overview & Use Case

This lab extends the appointment system from Lab 1 into a full **Medical Claims Processing & Patient Enrichment System**. When a new appointment is confirmed the system enriches it by calling external REST and SOAP services, caches frequently used data, handles transient failures with retry logic and message expiry, and submits completed claims to an external billing service.

Use Case Summary	
Business goal	Enrich each appointment with demographics, doctor availability, and insurance status before persisting and submitting the claim
External REST	Patient Lookup Service — returns demographics, allergies, medical aid reference
External SOAP	Insurance Validation Service — validates policy and returns coverage limit
External SOAP	Billing / Claims Service — receives enriched claim, returns confirmation
Cache	Patient and insurance data cached 5 min; doctor slot cache 60 sec (ITX Shared Cache)
Retry	Failed external calls retry up to 3 times with exponential backoff (5s, 15s, 45s)
Message expiry	MQMD.Expiry = 900 (90 s); messages exceeding TTL routed to APPT.EXPIRED.Q

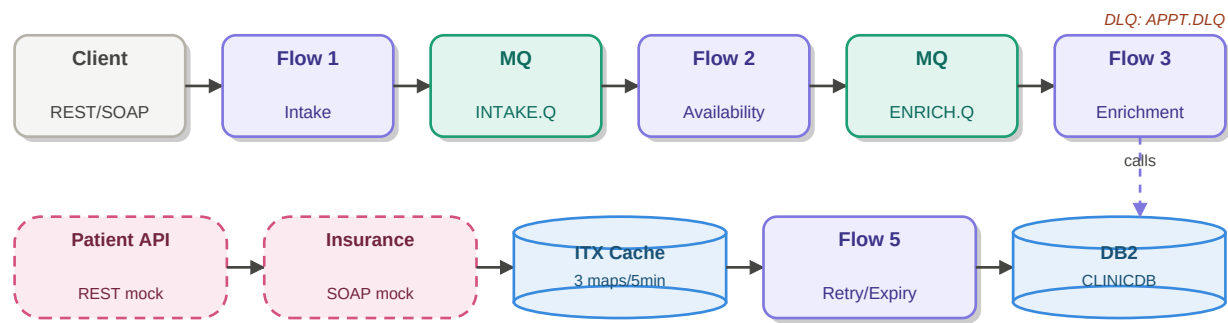
Flow Summary

#	Flow	Key Feature
1	Appointment Intake	Validates, stamps MQMD.Expiry=900, puts to APPT.INTAKE.Q
2	Doctor Availability	REST lookup with DoctorSlotCache (60 s TTL)
3	Patient Enrichment Orchestrator	Cache-aside for Patient + Insurance; merges; writes DB
4	Insurance Validation	External SOAP sub-flow; stores result in InsuranceCache
5	Retry & Expiry	Enforces TTL; exponential backoff; max 3 retries before DLQ
6	Claims Submission	Builds SOAP claim; calls external billing service; records confirmation

Part 2 — Architecture & Data Flow

The diagram shows all six flows, external services, the ITX cache, and DB2. Dashed lines indicate asynchronous or conditional paths (cache reads, retry loops, expiry routing).

Overall System Architecture



Data Flow Sequence	
Step 1	Client → Flow 1: validate + stamp expiry → APPT.INTAKE.Q
Step 2	Flow 2: dequeue → check DoctorSlotCache → call REST API on miss → APPT.ENRICH.Q
Step 3	Flow 3: dequeue → check PatientCache + InsuranceCache
Step 4	Cache MISS: call Patient REST + Insurance SOAP; store results in cache
Step 5	Flow 3: merge all data → INSERT ENRICHED_APPOINTMENTS → put to APPT.CLAIMS.Q
Step 6	Flow 6: build SOAP claim → call billing service → INSERT CLINIC.CLAIMS
Retry	Any external call failure → APPT.RETRY.Q → Flow 5 manages retry + expiry

Part 3 — External Services — Mock Setup

Use WireMock to simulate all three external services. Start the mock server before building any flows.

Patient Lookup REST stub

WireMock — Patient Lookup response

```
# GET /patients/{id} → 200 JSON
{  "patientId":"PT-0042", "fullName":"Thabo Nkosi",
   "dob":"1990-03-15", "bloodType":"O+",
   "allergies":["Penicillin", "Sulfonamides"],
   "medicalAidNumber":"MA-88712", "lastVisit":"2024-01-10" }
```

Insurance Validation SOAP stub

WireMock — Insurance Validation SOAP response

```
# POST /insurance/validate SOAPAction: validatePolicy
<ValidatePolicyResponse>
  <PolicyNumber>POL-447812</PolicyNumber>
  <Status>ACTIVE</Status>
  <CoverageLimit>50000.00</CoverageLimit>
  <ExpiryDate>2025-12-31</ExpiryDate>
</ValidatePolicyResponse>
```

Billing Claims SOAP stub

WireMock — Billing Claims SOAP response

```
# POST /billing/claims
<SubmitClaimResponse>
  <ConfirmationNo>CLM-20240413-9981</ConfirmationNo>
  <Status>SUBMITTED</Status>
  <EstimatedReimbursement>4200.00</EstimatedReimbursement>
</SubmitClaimResponse>
```

- Start WireMock: `java -jar wiremock.jar --port 8443 --root-dir ./mocks`
- Verify REST: `curl http://localhost:8443/patients/PT-0042`
- Verify SOAP: Postman POST with SOAPAction: validatePolicy header

Part 4 — Database Schema

DDL — all four tables for Lab 2

```
CREATE TABLE CLINIC.ENRICHED_APPOINTMENTS (
  APPT_ID          VARCHAR(30) NOT NULL PRIMARY KEY,
  PATIENT_ID       VARCHAR(20), PATIENT_NAME  VARCHAR(100),
  BLOOD_TYPE       VARCHAR(5),  ALLERGIES    VARCHAR(300),
  MEDICAL_AID_NO   VARCHAR(30), DOCTOR       VARCHAR(100),
  APPT_DATE        DATE,        APPT_TIME    TIME,
  DEPT             VARCHAR(50), INSURANCE_POLICY VARCHAR(30),
  INSURANCE_STATUS VARCHAR(20), COVERAGE_LIMIT DECIMAL(10,2),
  DOCTOR_CONFIRMED CHAR(1) DEFAULT 'N',
  STATUS           VARCHAR(20)  DEFAULT 'ENRICHED',
  CREATED_AT       TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE CLINIC.CLAIMS (
  CLAIM_ID         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  APPT_ID          VARCHAR(30) REFERENCES CLINIC.ENRICHED_APPOINTMENTS,
  CONFIRMATION_NO   VARCHAR(40), SUBMISSION_STATUS VARCHAR(20),
  EST_REIMBURSEMENT DECIMAL(10,2),
  SUBMITTED_AT     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE CLINIC.RETRY_LOG (
  LOG_ID          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  MESSAGE_ID       VARCHAR(50), TARGET_SVC VARCHAR(100),
  ATTEMPT_NO       INTEGER,      ERROR_CODE VARCHAR(20),
  RETRY_AT         TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE CLINIC.CACHE_LOG (
  LOG_ID          INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  CACHE_KEY        VARCHAR(100), HIT CHAR(1),
  LOOKUP_AT        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Part 5 — IBM MQ Queue Design

MQSC — all queues for Lab 2

```
runmqsc CLINIC.QM
DEFINE QLOCAL('APPT.INTAKE.Q')    MAXDEPTH(5000) MSGDLVSQ(FIFO);
DEFINE QLOCAL('APPT.ENRICH.Q')    MAXDEPTH(5000);
DEFINE QLOCAL('APPT.RETRY.Q')     MAXDEPTH(2000);
DEFINE QLOCAL('APPT.EXPIRED.Q')   MAXDEPTH(1000);
DEFINE QLOCAL('APPT.CLAIMS.Q')    MAXDEPTH(5000);
DEFINE QLOCAL('APPT.DLQ')         MAXDEPTH(2000);
ALTER QMGR DEADQ('APPT.DLQ');
END
```

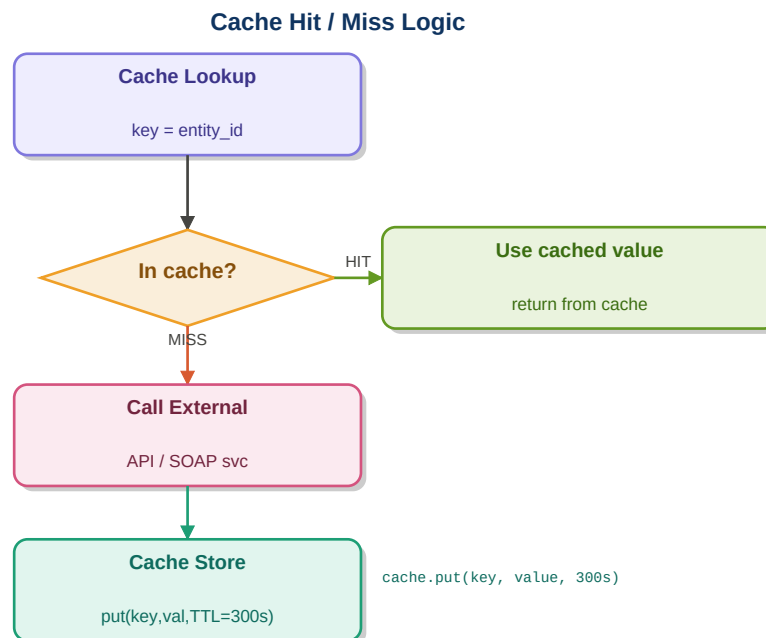
Queue	Fed by	Consumed by	Purpose
APPT.INTAKE.Q	Flow 1	Flow 2	New appointment messages
APPT.ENRICH.Q	Flow 2	Flow 3	Doctor-verified messages
APPT.RETRY.Q	Flows 2–4	Flow 5	Failed external calls
APPT.CLAIMS.Q	Flow 3	Flow 6	Enriched records ready for billing
APPT.EXPIRED.Q	Flow 5	—	TTL-exceeded messages (audit only)
APPT.DLQ	All flows	—	Unrecoverable failures

Part 6 — ITX Shared Cache Configuration

server.conf.yaml — ITX cache maps

```
# server.conf.yaml
SharedCache:
  enabled: true
  cacheManager: ClinicCacheManager
  maps:
    - name: PatientCache
      timeToLive: 300      # 5 minutes
      maxEntries: 1000
    - name: InsuranceCache
      timeToLive: 300      # 5 minutes
      maxEntries: 500
    - name: DoctorSlotCache
      timeToLive: 60       # 1 minute
      maxEntries: 200
```

Cache hit / miss logic



Cache Map	Key Pattern	TTL	Max entries
PatientCache	PT:{patientId}	300 s	1000
InsuranceCache	INS:{patientId}:{date}	300 s	500
DoctorSlotCache	DR:{doctorId}:{date}:{time}	60 s	200

Part 7 — WSDL: Insurance Validation Service

InsuranceValidation.wsdl

```
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="InsuranceValidationService"
  targetNamespace="http://cityclinic.co.za/insurance/wsdl"
  xmlns="http://schemas.xmlsoap.org/wsdl/"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
  xmlns:tns="http://cityclinic.co.za/insurance/wsdl"
  xmlns:ins="http://cityclinic.co.za/insurance">
  <types>
    <xsd:schema targetNamespace="http://cityclinic.co.za/insurance">
      <xsd:element name="ValidatePolicyRequest">
        <xsd:complexType><xsd:sequence>
          <xsd:element name="PatientID" type="xsd:string"/>
          <xsd:element name="MedicalAidNo" type="xsd:string"/>
          <xsd:element name="ServiceDate" type="xsd:date"/>
          <xsd:element name="ProcedureCode" type="xsd:string" minOccurs="0"/>
        </xsd:sequence></xsd:complexType>
      </xsd:element>
      <xsd:element name="ValidatePolicyResponse">
        <xsd:complexType><xsd:sequence>
          <xsd:element name="PolicyNumber" type="xsd:string"/>
          <xsd:element name="Status" type="xsd:string"/>
          <xsd:element name="CoverageLimit" type="xsd:decimal"/>
          <xsd:element name="ExpiryDate" type="xsd:date"/>
        </xsd:sequence></xsd:complexType>
      </xsd:element>
    </xsd:schema>
  </types>
  <message name="ValidatePolicyRequest">
    <part name="parameters" element="ins:ValidatePolicyRequest"/></message>
  <message name="ValidatePolicyResponse">
    <part name="parameters" element="ins:ValidatePolicyResponse"/></message>
  <portType name="InsurancePort">
    <operation name="validatePolicy">
      <input message="tns:ValidatePolicyRequest"/>
      <output message="tns:ValidatePolicyResponse"/>
    </operation>
  </portType>
  <binding name="InsuranceSoapBinding" type="tns:InsurancePort">
    <soap:binding style="document" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="validatePolicy">
      <soap:operation soapAction="validatePolicy"/>
      <input><soap:body use="literal"/></input>
      <output><soap:body use="literal"/></output>
    </operation>
  </binding>
  <service name="InsuranceValidationService">
    <port name="InsurancePort" binding="tns:InsuranceSoapBinding">
      <soap:address location="http://localhost:8443/insurance/validate"/>
    </port>
  </service>
</definitions>
```

Part 8 — OpenAPI: Patient Lookup Service

PatientLookup.yaml — OpenAPI 3.0

```

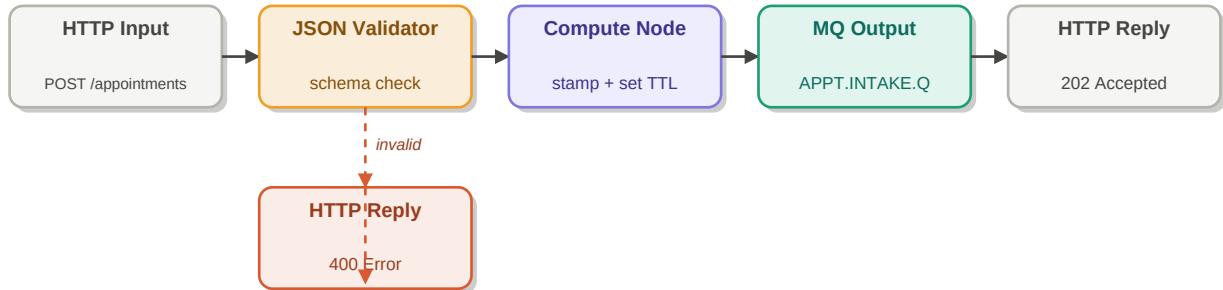
openapi: '3.0.3'
info:
  title: Patient Lookup Service
  version: '1.0.0'
servers:
  - url: http://localhost:8443
paths:
  /patients/{patientId}:
    get:
      summary: Get patient demographics
      parameters:
        - name: patientId
          in: path
          required: true
          schema: { type: string }
      responses:
        '200': { description: Patient record }
        '404': { description: Not found }
        '503': { description: Unavailable – triggers retry }
  /doctors/{doctorId}/availability:
    get:
      summary: Check slot availability
      parameters:
        - name: doctorId
          in: path
          required: true
          schema: { type: string }
        - name: date
          in: query
          required: true
          schema: { type: string, format: date }
        - name: time
          in: query
          required: true
          schema: { type: string }
      responses:
        '200': { description: Availability status }
components:
  schemas:
    PatientRecord:
      properties:
        patientId: { type: string }
        fullName: { type: string }
        dob: { type: string, format: date }
        bloodType: { type: string }
        allergies: { type: array, items: { type: string } }
        medicalAidNumber: { type: string }
    AvailabilityResponse:
      properties:
        available: { type: boolean }
        slotId: { type: string }

```

Part 9 — Flow 1: Appointment Intake Flow

Entry point. Receives appointment bookings via REST, validates the payload, stamps each message with a creation timestamp and sets **MQMD.Expiry = 900** (90 seconds), then puts to **APPT.INTAKE.Q**.

Flow 1 — Appointment Intake



Node	Configuration
HTTP Input	POST /appointments; port 7800
JSON Validator	Validates AppointmentRequest schema; invalid → HTTP 400
Compute Node	Sets MQMD.Expiry=900; adds meta.createdAt; meta.messageId; meta.retryCount=0
MQ Output	Queue: APPT.INTAKE.Q; Persistence: MQPER_PERSISTENT
HTTP Reply	Status 202; body: {message_id, status:'QUEUED'}

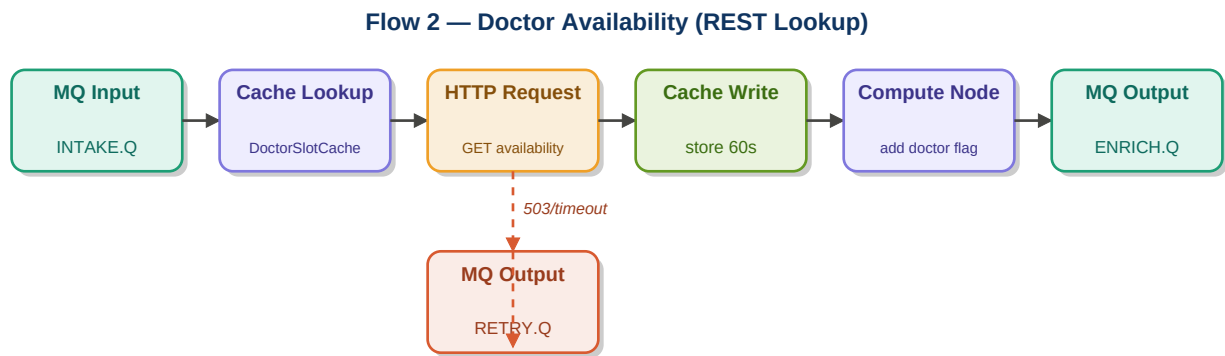
ESQL — StampAndSetExpiry compute node

```

-- ESQL: StampAndSetExpiry
SET OutputLocalEnvironment.MQ.MQMD.Expiry = 900; -- 90 seconds
SET OutRoot.JSON.Data.meta.createdAt =
  CAST(CURRENT_TIMESTAMP AS CHAR FORMAT 'yyyy-MM-ddTHH:mm:ssZ');
SET OutRoot.JSON.Data.meta.messageId =
  'MSG-' || CAST(CURRENT_TIMESTAMP AS CHAR FORMAT 'yyyymmddHHmmssSSS');
SET OutRoot.JSON.Data.meta.retryCount = 0;
  
```

Part 10 — Flow 2: Doctor Availability Flow (REST Lookup)

Dequeues from **APPT.INTAKE.Q**, checks **DoctorSlotCache** (60 s TTL), calls external REST API on miss, stores result, and forwards to **APPT.ENRICH.Q**.



Node	Configuration
MQ Input	Queue: APPT.INTAKE.Q
Cache Lookup	Map: DoctorSlotCache; Key: {doctorId}:{date}:{time}; HIT → skip HTTP call
HTTP Request	GET http://localhost:8443/doctors/{id}/availability?date=&time=; Timeout: 10 s
Cache Write	On 200: cache.put(key, responseBody, 60)
Compute Node	Sets payload.doctorConfirmed=(available==true); adds slotId
MQ Output (fail)	Queue: APPT.RETRY.Q; sets meta.targetService='DOCTOR_API'

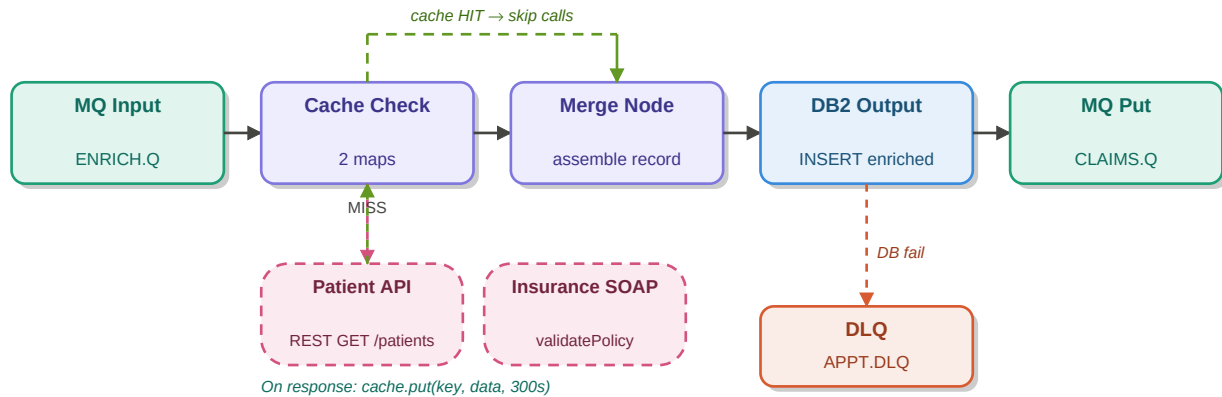
ESQL — DoctorSlotCache lookup

```
-- Cache lookup ESQL
DECLARE cacheKey CHAR 'DR:' || appt.doctor || ':' || appt.date || ':' || appt.time;
DECLARE cached CHAR cache.get('DoctorSlotCache', cacheKey);
IF cached IS NOT NULL THEN
  SET Environment.DoctorSlot = CAST(cached AS ROW);
  PROPAGATE TO TERMINAL 'out_hit';
  RETURN FALSE;
END IF;
-- cache MISS: fall through to HTTP Request node
```

Part 11 — Flow 3: Patient Enrichment Orchestrator

Core enrichment flow. Checks PatientCache and InsuranceCache; calls external services on misses; merges all data into a complete enriched record; writes to DB2; puts to APPT.CLAIMS.Q.

Flow 3 — Patient Enrichment Orchestrator



Node	Configuration
MQ Input	Queue: APPT.ENRICH.Q
Cache Check	Checks PatientCache (PT:{id}) and InsuranceCache (INS:{id}:{date}); HIT → bypass both calls
HTTP Request	GET /patients/{id}; Timeout 8 s; 503 → APPT.RETRY.Q with targetService=PATIENT_API
SOAP Request	WSDL: InsuranceValidation.wsdl; op: validatePolicy; Timeout 12 s
Merge Node	Combines appointment + doctorAvailability + patientRecord + insuranceStatus
DB2 Output	Table: CLINIC.ENRICHED_APPOINTMENTS; also puts to APPT.CLAIMS.Q

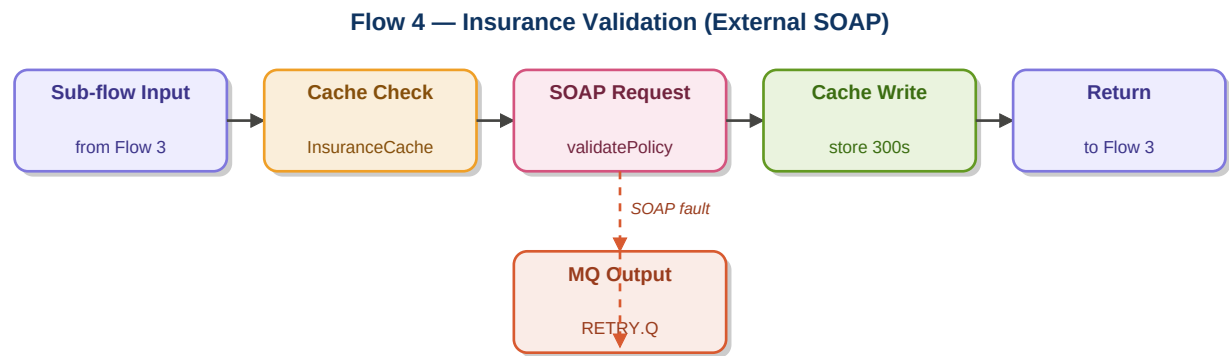
ESQL — AssembleEnrichedRecord

```

-- Merge Node ESQL: AssembleEnrichedRecord
SET Out.appt_id      = appt.appointment_id;
SET Out.patient_name = Environment.Patient.fullName;
SET Out.blood_type   = Environment.Patient.bloodType;
SET Out.allergies     = Environment.Patient.allergies;
SET Out.insurance_policy = Environment.Insurance.PolicyNumber;
SET Out.insurance_status = Environment.Insurance.Status;
SET Out.coverage_limit = Environment.Insurance.CoverageLimit;
SET Out.doctor_confirmed = appt.doctorConfirmed;
SET Out.status        = 'ENRICHED';
  
```

Part 12 — Flow 4: Insurance Validation Flow (External SOAP)

Dedicated SOAP consumer flow called as a sub-flow from Flow 3. Checks **InsuranceCache** first; on miss calls the external insurance WSDL; stores response in cache for 300 s.



SOAP payload to insurance service

```
<!-- SOAP Request to Insurance Validation service -->
<ins:ValidatePolicyRequest>
  <ins:PatientID>PT-0042</ins:PatientID>
  <ins:MedicalAidNo>MA-88712</ins:MedicalAidNo>
  <ins:ServiceDate>2024-04-15</ins:ServiceDate>
  <ins:ProcedureCode>GP-CONSULT-001</ins:ProcedureCode>
</ins:ValidatePolicyRequest>
```

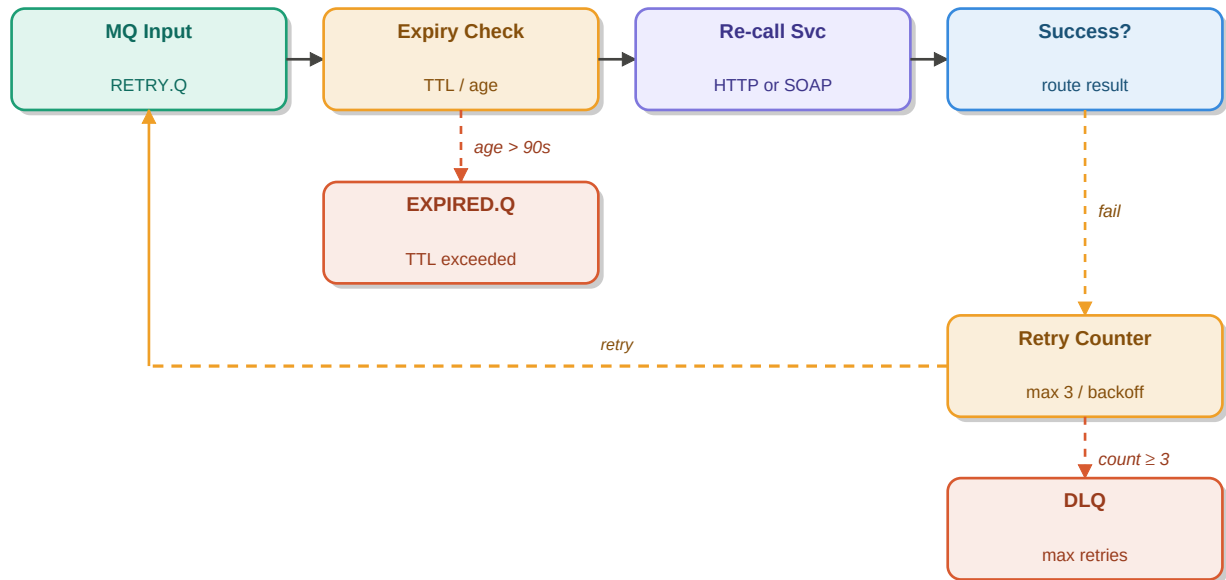
ESQL — InsuranceCache write

```
-- Cache write after successful SOAP response
DECLARE insKey CHAR 'INS:' || patientId || ':' || serviceDate;
CALL cache.put('InsuranceCache', insKey, CAST(soapResponse AS CHAR), 300);
```

Part 13 — Flow 5: Retry & Message Expiry Flow

All failed external calls land on **APPT.RETRY.Q** with an incremented retry counter. Flow 5 enforces both MQMD-level expiry and an application-level TTL check, then re-attempts the failing call with exponential backoff.

Flow 5 — Retry & Message Expiry Flow



MQMD.Expiry=900 (90 s) set at intake. App-level TTL check performed in Expiry Check node.

Concept	Implementation
MQMD.Expiry	Set to 900 (90 s) at intake; MQ auto-expires before dequeue to APPT.EXPIRED.Q
App TTL check	Compute node: if (now - meta.createdAt) > 90 s → route to APPT.EXPIRED.Q
Retry counter	meta.retryCount incremented per attempt; if >= 3 → APPT.DLQ
Backoff	Retry 1: 5 s Retry 2: 15 s Retry 3: 45 s (CALL sleep(backoff))
Target service	meta.targetService identifies which call failed (PATIENT_API, INSURANCE_SOAP, DOCTOR_API)

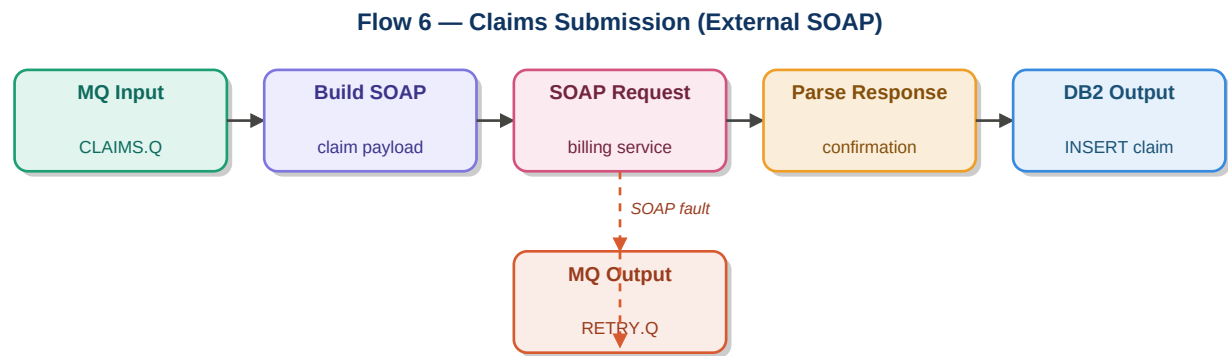
ESQL — CheckExpiryAndRetryCount

```

-- CheckExpiryAndRetryCount ESQL
DECLARE ageSeconds INTEGER
CAST(INTERVAL(CURRENT_TIMESTAMP -
CAST(meta.createdAt AS TIMESTAMP FORMAT 'yyyy-MM-dd'T'HH:mm:ssz'))
SECOND(6) AS INTEGER);
IF ageSeconds > 90 THEN
SET OutLocalEnv.MQ.MQMD.DestinationData[1].queueName = 'APPT.EXPIRED.Q';
PROPAGATE TO TERMINAL 'out_expired'; RETURN FALSE;
END IF;
DECLARE retries INTEGER InputRoot.JSON.Data.meta.retryCount;
IF retries >= 3 THEN
SET OutLocalEnv.MQ.MQMD.DestinationData[1].queueName = 'APPT.DLQ';
PROPAGATE TO TERMINAL 'out_dlq'; RETURN FALSE;
END IF;
SET OutRoot.JSON.Data.meta.retryCount = retries + 1;
DECLARE backoff INTEGER POWER(3, retries) * 5; -- 5,15,45 seconds
CALL sleep(backoff);
  
```

Part 14 — Flow 6: Claims Submission Flow (External SOAP)

After Flow 3 writes the enriched record and puts to **APPT.CLAIMS.Q**, Flow 6 picks it up, builds a SOAP claim, calls the external billing service, and records the confirmation in **CLINIC.CLAIMS**.



SOAP claim payload — submitted to billing service

```
<bil:SubmitClaimRequest>
  <bil:AppointmentID>APT-20240413-001</bil:AppointmentID>
  <bil:PatientID>PT-0042</bil:PatientID>
  <bil:PolicyNumber>POL-447812</bil:PolicyNumber>
  <bil:CoverageLimit>50000.00</bil:CoverageLimit>
  <bil:DoctorID>DR-PATEL-001</bil:DoctorID>
  <bil:ServiceDate>2024-04-15</bil:ServiceDate>
  <bil:ProcedureCode>GP-CONSULT-001</bil:ProcedureCode>
</bil:SubmitClaimRequest>
```

DB2 INSERT after billing confirmation

```
INSERT INTO CLINIC.CLAIMS (APPT_ID, CONFIRMATION_NO, SUBMISSION_STATUS, EST_REIMBURSEMENT)
VALUES (?appt_id, ?ConfirmationNo, ?Status, ?EstimatedReimbursement);
```


Part 15 — Cache Design & ESQL Patterns

ESQL — cache-aside pattern

```
-- Full cache-aside pattern (ESQL)
DECLARE cacheKey CHAR 'PT:' || patientId;
DECLARE cached CHAR cache.get('PatientCache', cacheKey);
IF cached IS NOT NULL THEN
  -- HIT: parse cached JSON into environment
  SET Environment.Patient = CAST(cached AS ROW FORMAT JSON);
  INSERT INTO Database.CLINIC.CACHE_LOG(CACHE_KEY,HIT) VALUES(cacheKey,'Y');
  PROPAGATE TO TERMINAL 'out_hit'; RETURN FALSE;
END IF;
-- MISS: log and continue to HTTP/SOAP node
INSERT INTO Database.CLINIC.CACHE_LOG(CACHE_KEY,HIT) VALUES(cacheKey,'N');
PROPAGATE TO TERMINAL 'out_miss'; RETURN FALSE;
```

ESQL — cache API reference

```
-- Cache write after successful external call
CALL cache.put('PatientCache', cacheKey, CAST(responseBody AS CHAR), 300);
-- Check key exists without retrieving
DECLARE exists BOOLEAN cache.contains('PatientCache', cacheKey);
-- Invalidate a stale entry (e.g. after patient data update)
CALL cache.remove('PatientCache', cacheKey);
-- Emergency flush of entire map
CALL cache.clear('PatientCache');
```

Part 16 — Retry & Expiry ESQL Patterns

ESQL — RouteToRetry error handler

```
-- RouteToRetry: called from error catch of any HTTP/SOAP node
CREATE COMPUTE MODULE RouteToRetry
CREATE FUNCTION main() RETURNS BOOLEAN
BEGIN
  SET OutputRoot = InputRoot; -- preserve original message
  SET OutRoot.JSON.Data.meta.targetService = 'PATIENT_API';
  SET OutRoot.JSON.Data.meta.lastError =
    CAST(FailureList.failure[1].insert.SQLSTATE AS CHAR);
  DECLARE rc INTEGER COALESCE(InRoot.JSON.Data.meta.retryCount, 0);
  SET OutRoot.JSON.Data.meta.retryCount = rc + 1;
  SET OutLocalEnv.MQ.MQMD.DestinationData[1].queueName = 'APPT.RETRY.Q';
  RETURN TRUE;
END;
END MODULE;
```

Attempt	Delay	Destination	Condition
Initial failure	Immediate	APPT.RETRY.Q	retryCount set to 1
Retry 1	5 seconds	APPT.RETRY.Q	retryCount becomes 2
Retry 2	15 seconds	APPT.RETRY.Q	retryCount becomes 3
Retry 3	45 seconds	APPT.DLQ	count >= 3: discard
TTL expired	N/A	APPT.EXPIRED.Q	age > 90 s at any point

Verification — expiry queue and retry log

```
-- Verify expiry queue depth
runmqsc CLINIC.QM
DISPLAY QLOCAL('APPT.EXPIRED.Q') CURDEPTH;
END

-- Expired + retry stats from DB
SELECT TARGET_SVC, COUNT(*) TOTAL, MAX(Attempt_No) MAX_ATTEMPTS
FROM CLINIC.RETRY_LOG
WHERE DATE(RETRY_AT) = CURRENT DATE
GROUP BY TARGET_SVC;
```

Part 17 — End-to-End Testing & Test Matrix

Postman + DB verification

```
# Test 1 – happy path
POST http://localhost:7800/cityclinic/v1/appointments
{
  "appointment_id":"APT-LAB2-001",
  "patient":{"id":"PT-0042","name":"Thabo Nkosi"},
  "doctor":"Dr. Ayasha Patel","department":"General Practice",
  "date":"2024-05-01","time":"09:30","reason":"Annual checkup" }

# Verify enriched record
SELECT APPT_ID,PATIENT_NAME,BLOOD_TYPE,INSURANCE_STATUS,COVERAGE_LIMIT,DOCTOR_CONFIRMED
FROM CLINIC.ENRICHED_APPOINTMENTS WHERE APPT_ID='APT-LAB2-001';

# Verify claim submitted
SELECT CONFIRMATION_NO,SUBMISSION_STATUS,EST_REIMBURSEMENT
FROM CLINIC.CLAIMS WHERE APPT_ID='APT-LAB2-001';
```

Scenario	Expected Result
Happy path — all services up	Enriched record in DB; claim confirmation in CLINIC.CLAIMS
Cache hit — same patientId twice	2nd call: no HTTP to Patient API; CACHE_LOG shows HIT
Patient API returns 503	Message on APPT.RETRY.Q; retried 3 times with backoff
Message age > 90 s (wait before Flow 5)	Message routed to APPT.EXPIRED.Q; logged in RETRY_LOG
Max retries exceeded (WireMock always 503)	After attempt 3 → APPT.DLQ
Insurance SOAP returns fault	SOAP fault caught → APPT.RETRY.Q with targetService=INSURANCE_SOAP
Billing SOAP timeout (WireMock delay 30s)	Flow 6 12s timeout → APPT.RETRY.Q
Duplicate appointment_id	DB UNIQUE violation → APPT.DLQ

Stretch Goals

- Replace ITX cache with Redis (ACE HTTP nodes → Redis REST API)
- Add circuit breaker: after 5 consecutive failures disable the external call for 60 s
- Build a monitoring flow: alert when APPT.RETRY.Q depth > 20
- Add mutual TLS to all external SOAP/REST calls using ACE SSL profiles
- Implement saga compensation: if claims fail after 3 retries, update appointment STATUS to CLAIM_FAILED
- Add a stats endpoint GET /admin/cache-stats → returns PatientCache hit rate from CACHE_LOG

Lab 2 complete. You have built 6 advanced flows: external REST and SOAP calls, a three-map ITX cache with configurable TTLs, retry with exponential backoff, MQMD and application-level expiry, and end-to-end claims processing via an external billing SOAP service.

Pre-requisite: complete Lab 1 first. Estimated time: 5 – 7 hours.